31 MONGOOSE SCHEMA DECORATORS — @Schema, @Prop, SchemaFactory

student.schema.ts uses a second family of decorators beyond the ones in theory/06-decorators.txt. Those describe HTTP/DI concerns (routes, injectable services). @Schema/@Prop describe a completely different concern: what a document looks like when it's stored in MongoDB.

```
@Schema()
export class Student {
  @Prop({ required: true })
  name!: string;

  @Prop({ required: true })
  age!: number;

  @Prop({ required: true, unique: true })
  email!: string;

  @Prop()
  course!: string;
}

export const StudentSchema = SchemaFactory.createForClass(Student);
```

@Schema() — CLASS DECORATOR

Marks the class as a Mongoose schema definition. Same mechanism as @Module()/@Controller() (theory/06) — it attaches metadata via reflect-metadata that something else reads later. On its own it does nothing at runtime; it just makes this class a valid input to SchemaFactory.createForClass() below.

Optional settings go inside the parentheses, e.g. @Schema({ timestamps: true }) auto-adds createdAt/updatedAt fields to every document.

@Prop() — PROPERTY DECORATOR

Marks one class field as a field that gets stored in MongoDB. The options object controls how Mongoose treats that field:

required: true	Mongoose refuses to save a document missing this field (a validation-layer check, enforced by Mongoose, not by MongoDB itself).
unique: true	Mongoose builds a unique INDEX on this field — MongoDB itself then rejects a second document with a duplicate value. This is a database-level guarantee, not just app-level validation.
default: value	If the field is omitted on insert, Mongoose fills in this value instead of leaving it unset.

```
type: SomeType       Usually inferred from the TS type automatically;
                      set explicitly for things TS can't express
```

```
                    (e.g. type: [String] for an array of strings,
                    or a nested schema/ref for relations).
```

A field with no @Prop() at all is just a normal TS class member — Mongoose ignores it and it is NOT stored in the database.

SchemaFactory.createForClass(Student)

@Schema()/@Prop() only attach METADATA (theory/06's core idea again — decorators label, they don't act). SchemaFactory.createForClass reads that metadata off the Student class and BUILDS an actual mongoose.Schema object from it — the real runtime schema Mongoose uses to validate documents, build indexes, and generate a Model.

The exported StudentSchema is what gets registered with a module:

```
MongooseModule.forFeature([{ name: Student.name, schema: StudentSchema }])
```

(see theory/30-mongodb-connection.txt, Approach 2, step 3) — this is the piece that turns "a decorated class" into "something Nest's DI container can hand you as an injectable Model."

THE `!` (DEFINITE ASSIGNMENT ASSERTION)

```
name!: string;
```

TypeScript's strictPropertyInitialization normally demands every class field be assigned in the constructor, or be typed optional (`name?: string`). Student has no constructor — Mongoose populates these fields at runtime, when it builds a document from data coming out of the database or an insert call, which TS's static analysis can't see. `!` tells the compiler "trust me, this will be set by the time anything reads it" and suppresses the "no initializer" error (TS2564) without changing the field to optional (which would wrongly allow `undefined` everywhere you use it).

Document / StudentDocument

```
export type StudentDocument = Student & Document;
```

`Student` alone only describes YOUR fields (name, age, email, course). `Document` (from the `mongoose` package) describes what Mongoose bolts onto every document it manages — `_id` , `.save()` , `.populate()` , timestamps, etc. Intersecting the two types gives you one type that has both: your fields AND Mongoose's built-in document methods. This is the type you inject as `Model<StudentDocument>` when writing a service that queries this collection.

HOW THIS DIFFERS FROM DTO DECORATORS (theory/13, theory/15)

Easy to conflate because both are "decorators on a class's properties," but they answer different questions at different points in a request's life:

DTO + class-validator	@Schema + @Prop

(@IsString(), @IsEmail())	(@Prop({ required: true }))
----------	----------
Validates data COMING IN from	Defines the shape of data STORED in
an HTTP request body, before	MongoDB, and Mongoose's own
it reaches your handler	validation when saving a document
(ValidationPipe, theory/09).	(independent of any HTTP request — also applies to seed scripts, etc).
Lives and dies with one	Describes a persistent database
request.	collection's structure.

A typical flow uses BOTH: a CreateStudentDto validates what the client sent, then a service maps that DTO's fields onto a call to `this.studentModel.create(dto)`, where the Student schema's own `required` / `unique` rules apply again at the database layer.

★ RULE OF THUMB

@Schema()/@Prop() are metadata, same as every other Nest/Mongoose decorator (**theory/06**) — they don't run logic themselves. SchemaFactory.createForClass is the step that actually turns that metadata into a working Mongoose Schema. `!` exists purely to satisfy TypeScript about fields Mongoose fills in outside any constructor. `Document` supplies the Mongoose-internal fields/methods your own class doesn't declare.

See also: **theory/06-decorators.txt**, **theory/30-mongodb-connection.txt**, **theory/13-dto.txt**, **theory/15-class-validator-and-class-transformer.txt**